

06 — Database Design

Overview

This module covers the complete art and science of relational database design — from the mathematical foundations of normalization theory to practical, production-ready schemas for common application domains. Every file is built around the principle: **understand the theory, then apply it with PostgreSQL's specific features**.

Files in This Module

File	Topic	Key Concepts
01_normalization_1nf_2nf_3nf.md	First three normal forms	Functional dependencies, anomaly elimination
02_bcnf_4nf_5nf.md	Advanced normal forms	BCNF, multi-valued deps, join deps
03_denormalization.md	Controlled redundancy	Materialized views, counter caches, star schema
04_entity_relationships.md	ER modeling	1:1, 1:N, M:N, junction tables, self-ref
05_schema_design_ecommerce.md	E-commerce schema	Complete DDL, 400+ lines, all constraints
06_schema_design_banking.md	Banking schema	Double-entry ledger, atomic transfers
07_schema_design_saas.md	SaaS multi-tenancy	RLS, schema-per-tenant, pattern comparison
08_schema_design_social_media.md	Social platform	Graph, feed generation, celebrity problem
09_schema_design_logistics.md	Logistics schema	Event sourcing, routing, geospatial
10_design_patterns.md	Reusable patterns	Soft deletes, audit, versioning, outbox

Normalization Quick Reference

Normal Form	Rule (simplified)	Fixes
1NF	Atomic values, unique rows	Non-atomic cells, repeating groups
2NF	Full dependency on entire PK	Partial dependencies (composite PK)
3NF	No non-key → non-key dependencies	Transitive dependencies
BCNF	Every FD has a superkey on left	Overlapping candidate keys
4NF	No independent multi-valued deps	Cartesian-product redundancy
5NF	Join deps implied by keys	Spurious joins in decomposition

Practical target: 3NF for OLTP, BCNF where overlapping keys exist, controlled denormalization where performance requires it.

Schema Design Decision Tree

Starting a new schema? Ask:

1. How many entities?
 - Draw your ER diagram first, before writing SQL
2. What are the relationships?
 - 1:N → FK on "many" side
 - M:N → junction table
 - 1:1 → shared PK or nullable FK
3. What are the functional dependencies?
 - List them explicitly before writing tables
4. What is the read:write ratio?
 - High writes → normalize
 - High reads, complex queries → consider materialized views
5. What domain is this?
 - Financial (banking) → double-entry, NUMERIC, no floats
 - Time-series → partitioning, BRIN indexes
 - Social → graph, feeds, counter caches
 - Multi-tenant → choose: RLS vs schema-per-tenant vs DB-per-tenant
 - E-commerce → catalog, inventory, order history snapshots
6. What are the compliance requirements?
 - GDPR → soft deletes, audit trail
 - Financial → immutable ledger, reconciliation
 - Healthcare → separate PII, encryption

Domain Schemas Quick Reference

E-commerce key entities:

categories → products → product_variants → inventory
customers → addresses
carts → cart_items
orders → order_items (price snapshot!)
payments

Banking key entities:

customers → accounts (typed)
transactions (high-level)
ledger_entries (double-entry, immutable)

SaaS key entities:

tenants → (RLS on all tables)
users → tenant_users (M:N)
projects → tasks

Social Media key entities:

- users → follows (self-ref M:N)
- posts → likes, comments, bookmarks
- feeds (pre-computed)
- notifications

Logistics key entities:

- locations (warehouses, hubs)
- vehicles → drivers
- shipments → shipment_events (event log)
- routes → route_stops
- inventory_stock → inventory_movements

Design Patterns Cheat Sheet

Pattern	Use when	PostgreSQL feature
Soft delete	Data recovery / compliance	deleted_at + partial index
Audit trail	Change tracking / compliance	AFTER trigger + JSONB
Row versioning	Concurrent edit detection	version column + WHERE check
History table	"Point in time" queries	valid_from/valid_to columns
Polymorphic	Comments on multiple entity types	Separate nullable FKs
State machine	Workflows with rules	transitions table + function
Outbox	Events must match DB state	Same-transaction table insert
Counter cache	Avoid COUNT(*) on hot paths	Trigger-maintained counter
Materialized view	Complex reporting queries	REFRESH CONCURRENTLY

Anti-Patterns to Avoid

- X EAV (entity-attribute-value) as string rows – use JSONB instead
- X Generic entity_type/entity_id FKs – no referential integrity
- X FLOAT for money – always NUMERIC(19,4)
- X Storing derived values without generated columns or triggers
- X "God table" with 50+ columns – break into related entities
- X No indexes on FK columns – slows DELETE on parent tables
- X Premature denormalization – profile first
- X OFFSET pagination – use cursor-based pagination
- X SERIAL instead of IDENTITY – use GENERATED AS IDENTITY

Learning Path

Foundation (read in order)

1. 01_normalization_1nf_2nf_3nf.md — the why of relational design
2. 04_entity_relationships.md — the vocabulary and mechanics
3. 02_bcnf_4nf_5nf.md — complete the normalization picture

Application

4. 05_schema_design_ecommerce.md — most concepts in one schema

5. `06_schema_design_banking.md` — financial integrity patterns
6. `10_design_patterns.md` — reusable building blocks

Advanced

7. `03_denormalization.md` — when to optimize
8. `07_schema_design_saas.md` — multi-tenancy at scale
9. `08_schema_design_social_media.md` — graph and feed design
10. `09_schema_design_logistics.md` — event sourcing and geospatial

Prerequisites

- `../05_PostgreSQL_Core/` — data types and constraints (required)
- Basic SQL: CREATE TABLE, SELECT, JOIN, INSERT

Next Modules

- `../07_Indexes/` — make these schemas fast
- `../08_Query_Optimization/` — write efficient queries against these schemas
- `../09_Transactions_Concurrency/` — keep multi-user data consistent
- `../14_Security/` — Row-Level Security and access control